CCE

Unit-3

Title : Hospital Patient Record Management Using Linked List

Problem Statement: A hospital needs an efficient system to maintain and manage patient records. The number of patients admitted to the hospital may change frequently, so using a fixed size array may waste memory or limit the number of records that can be stored.

The proposed system uses a singly linked list to store patient records dynamically. Each node represent one patient and contains details such as Patient ID, Name, Age, and Disease, along with a pointer to the next patient record.

Objectives : - To understand the concept and working of singly linked list.

- To implement a hospital patient record system using linked list

- To store patient information using dynamic memory allocation.

- To perform insertion of new patient records.

- To perform deletion of existing patient records

- To perform searching of patient records using Patient ID.

Algorithm: step1 : Start the program.
step2 : Define a Patient node containing:
- Patient ID
- Patient Name
- Age
- Disease
- Pointer to the next node.
step3 : Initialize head = NULL to indicate that the linked list is empty.
step4 : Display the menu:
- Insert Patient
- Display Patients
- Search Patient
- Delete Patient
- Exit

Step5: Insert Patient
- Create a new node dynamically.
- Read Patient ID, Name, Age and Disease.
- Set the new node's next pointer to NULL.
- If the list is empty, make the new node the head.
- Otherwise, traverse to the last node.
- Link the last node to new node.
- Display successful insertion message.

Step6: Display Patients
- Check whether head == NULL.
- If yes, display "No patients records found".
- Otherwise, start from head.
- Display details of each patient.
- move to next node using next pointer.
- Continue until NULL is reached.

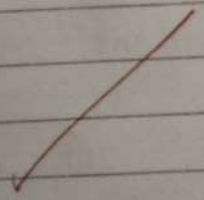
Step7: Search Patient
- Ask the uger to enter patient ID.
- Start traversal from head.
- Compare each nodes patient ID with entered ID.
- If ID matches, display patient's details.
- If NULL is reached without match, display "Patient not found".

Steps: Delete Patient

- Ask user to enter Patient ID to delete
- Traverse linked list & search for patient.
- Maintain pointers to current & remove previous node.
- Free the memory occupied by the deleted node.
- Display a successful deletion message.
- If patient is not found, display "Patient not found".

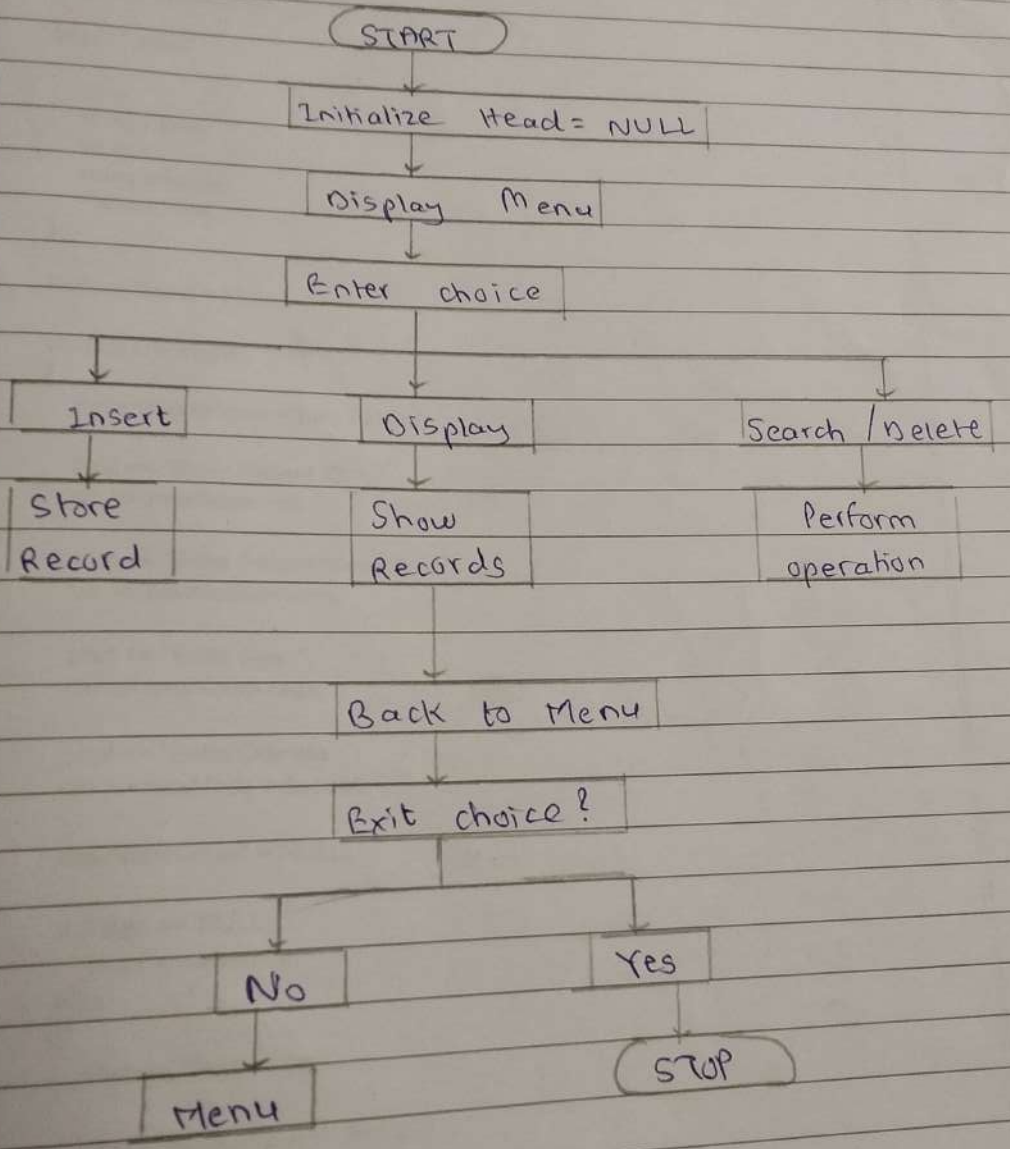
Step9: Repeat the menu until user selects Exit.

Step10: Stop the program.



Page No.
Date

Flowchart:



```
#include <iostream>
#include <string>
using namespace std;

struct Patient
{
   int id;
   string name;
   int age;
   string disease;
   Patient *next;
};

Patient *head = NULL;

void insertPatient()
{
   Patient *newNode = new Patient;

   cout << "Enter Patient ID: ";
   cin >> newNode->id;

   cout << "Enter Patient Name: ";
   cin >> newNode->name;

   cout << "Enter Age: ";
   cin >> newNode->age;

   cout << "Enter Disease: ";
   cin >> newNode->disease;

   newNode->next = NULL;

   if (head == NULL)
      head = newNode;
   else
   {
      Patient *temp = head;

      while (temp->next != NULL)
         temp = temp->next;

      temp->next = newNode;
   }

   cout << "Patient record inserted successfully.\n";
}
```

```
void displayPatients()
{
    if (head == NULL)
    {
        cout << "No patient records found.\n";
        return;
    }

    Patient *temp = head;

    cout << "\nPatient Records:\n";

    while (temp != NULL)
    {
        cout << "ID: " << temp->id << endl;
        cout << "Name: " << temp->name << endl;
        cout << "Age: " << temp->age << endl;
        cout << "Disease: " << temp->disease << endl;
        cout << "----------\n";

        temp = temp->next;
    }
}

void searchPatient()
{
    int id;

    cout << "Enter Patient ID to search: ";
    cin >> id;

    Patient *temp = head;

    while (temp != NULL)
    {
        if (temp->id == id)
        {
            cout << "Patient Found!\n";
            cout << "Name: " << temp->name << endl;
            cout << "Age: " << temp->age << endl;
            cout << "Disease: " << temp->disease << endl;
            return;
        }

        temp = temp->next;
    }

    cout << "Patient not found.\n";
```

```
}

void deletePatient()
{
    int id;

    cout << "Enter Patient ID to delete: ";
    cin >> id;

    Patient *temp = head;
    Patient *prev = NULL;

    while (temp != NULL && temp->id != id)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        cout << "Patient not found.\n";
        return;
    }

    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;

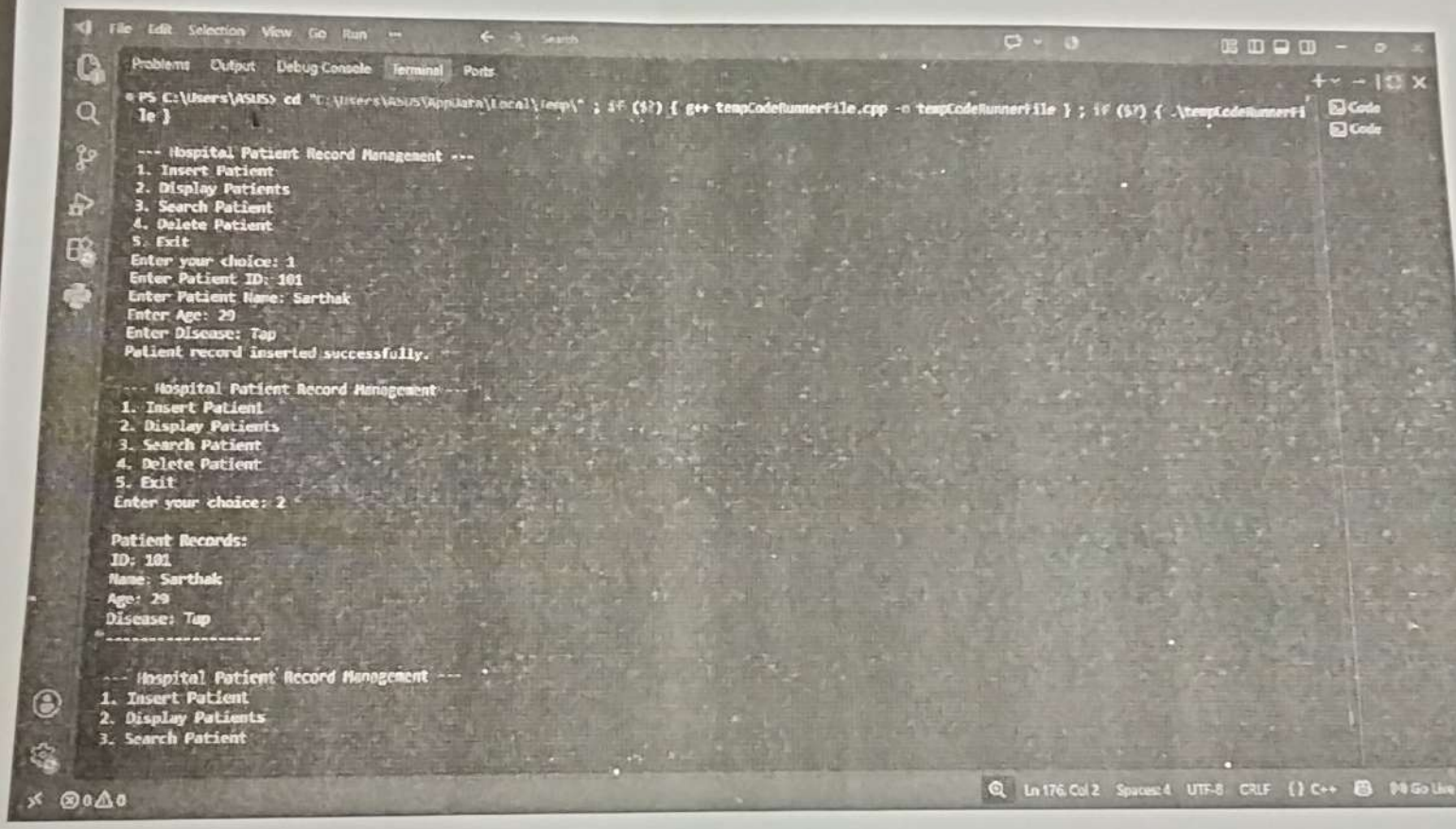
    delete temp;

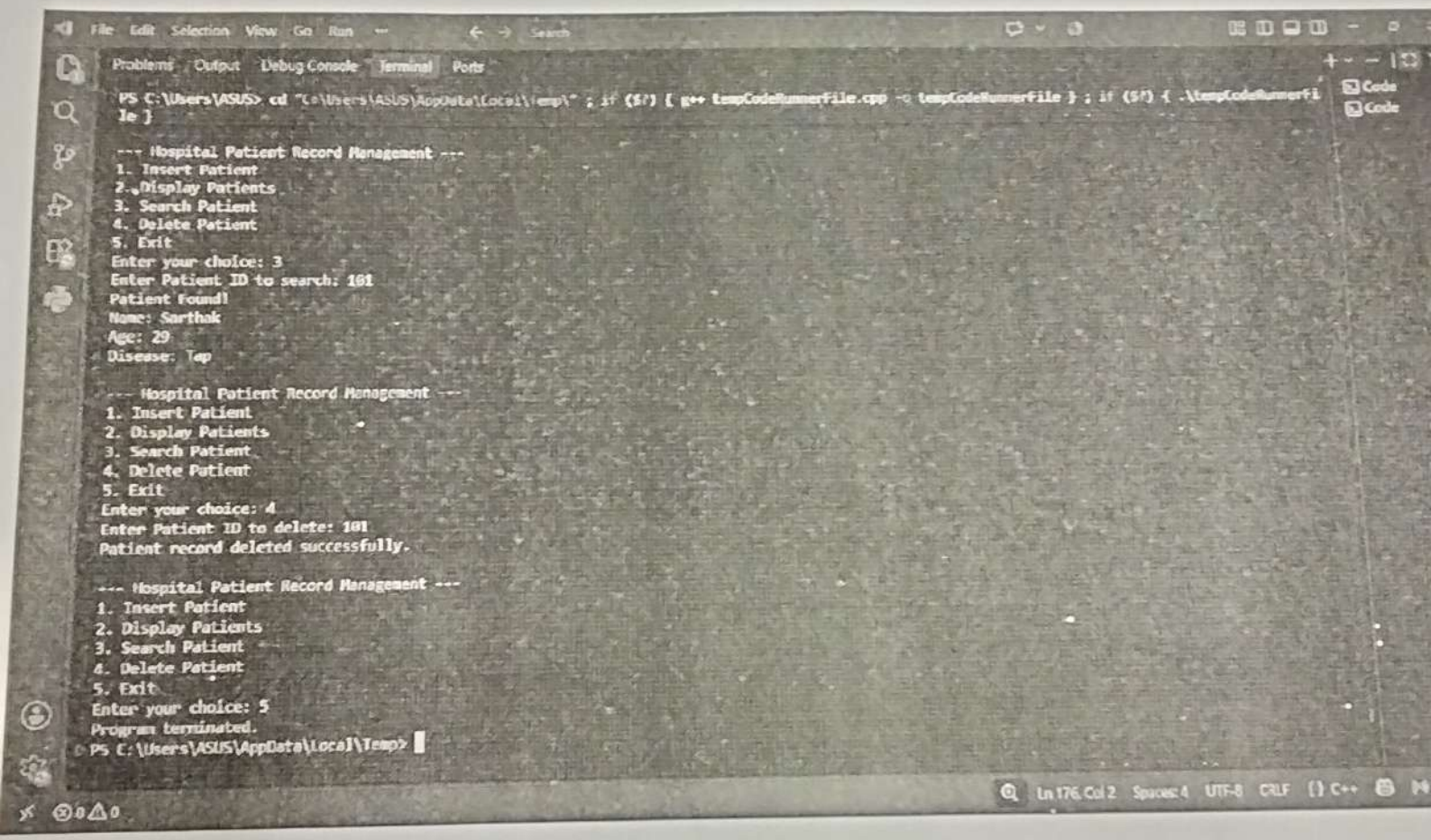
    cout << "Patient record deleted successfully.\n";
}

int main()
{
    int choice;

    do
    {
        cout << "\n--- Hospital Patient Record Management ---\n";
        cout << "1. Insert Patient\n";
        cout << "2. Display Patients\n";
        cout << "3. Search Patient\n";
        cout << "4. Delete Patient\n";
        cout << "5. Exit\n";

        cout << "Enter your choice: ";
```


File Edit Selection View Go Run
Problems Output Debug Console Terminal Ports
PS C:\Users\ASUS> cd "C:\Users\ASUS\AppData\Local\Temp\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
--- Hospital Patient Record Management ---
1. Insert Patient
2. Display Patients
3. Search Patient
4. Delete Patient
5. Exit
Enter your choice: 1
Enter Patient ID: 101
Enter Patient Name: Sarthak
Enter Age: 29
Enter Disease: Tap
Patient record inserted successfully.
--- Hospital Patient Record Management ---
1. Insert Patient
2. Display Patients
3. Search Patient
4. Delete Patient
5. Exit
Enter your choice: 2
Patient Records:
ID: 101
Name: Sarthak
Age: 29
Disease: Tap
--- Hospital Patient Record Management ---
1. Insert Patient
2. Display Patients
3. Search Patient
Ln 176, Col 2 Spaces: 4 UTF-8 CRLF C++


File Edit Selection View Go Run
Problems Output Debug Console Terminal Ports
PS C:\Users\ASUS> cd "c:\Users\ASUS\AppData\Local\Temp\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
--- Hospital Patient Record Management ---
1. Insert Patient
2. Display Patients
3. Search Patient
4. Delete Patient
5. Exit
Enter your choice: 3
Enter Patient ID to search: 101
Patient found!
Name: Sarthak
Age: 29
Disease: Tap
--- Hospital Patient Record Management ---
1. Insert Patient
2. Display Patients
3. Search Patient
4. Delete Patient
5. Exit
Enter your choice: 4
Enter Patient ID to delete: 101
Patient record deleted successfully.
--- Hospital Patient Record Management ---
1. Insert Patient
2. Display Patients
3. Search Patient
4. Delete Patient
5. Exit
Enter your choice: 5
Program terminated.
PS C:\Users\ASUS\AppData\Local\Temp>
Ln 176, Col 2 Spaces: 4 UTF-8 CRLF {} C++